



Contents lists available at ScienceDirect

Journal of King Saud University – Computer and Information Sciences

journal homepage: www.sciencedirect.com

An approach based on genetic algorithms and neural networks for QoS-aware IoT services composition

Rabah Boucetti^{a,*}, Ouassila Hioual^c, Sofiane Mounine Hemam^b^a ICOSI Laboratory, Abbas Laghrour University, Khenchela 40004, Algeria^b Abbas Laghrour University, Khenchela 40004, Algeria^c LIRE Laboratory of Constantine, Abbas Laghrour University, Khenchela 40004, Algeria

ARTICLE INFO

Article history:

Received 21 September 2021

Revised 25 December 2021

Accepted 11 February 2022

Available online 26 February 2022

Keywords:

IoT Service

IoT Service Composition

Quality of Service decomposition

Neural Network

Genetic Algorithm

ABSTRACT

In the Internet of Things (IoT) environment, the services provided by the connected objects are published as services through the web. This allows to machines to interact between them and, makes the IoT services composition possible. However, the vast proliferation of smart object generates services with the same functionalities but different in terms of quality of services (QoS) proprieties. This makes the satisfaction of the user requirements often complex and a NP-hard problem. Indeed, respecting the QoS constraints (user preferences in terms of QoS) is a challenge, due to the high number of candidate services for the composition. This challenge consists of selecting the most appropriate services so that the composite service must meet both the functional and the non-functional requirements of the user. To deal with this challenge, we propose an approach based on Genetic Algorithms (GA) and Neural Networks (NN) for QoS-aware IoT Services Composition in the context of large-scale environments. The combination between GA and NN allows finding the quasi-optimal IoT service composition. This latter is based on global QoS optimization. To reach this objective, the QoS intervals are decomposed into M QoS-levels to engage them into the theoretical composition. Then, the proposed first GA is used to obtain the ideal theoretical composition with an overall QoS optimization. Afterward, the proposed NN is used to eliminate the inappropriate concrete IoT services, and retain only the services having the same categories of the atomic theoretical services composing the ideal theoretical composition. This allows us to optimize the search space as well as the execution time. Finally, we apply the second GA on the concrete services of the retained categories, in order to obtain the IoT service concrete composition with an overall QoS optimization. The simulation results show that the proposed approach has the best composition time, the best Hypervolume indicator and the best compositional optimality compared to SC-FLA, Improved GA and MGA approaches. On another side, it has almost the same performances compared to TS-QCA and, it finds the near-optimal composition in a very short time compared to PSA, which is an optimal approach. Thus, the obtained results show the effectiveness of our approach.

© 2022 The Authors. Published by Elsevier B.V. on behalf of King Saud University. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

1. Introduction

In recent years, the advancements in the Internet of Things (IoT) technology has gained a lot of popularity (Gupta et al., 2021). This emerging technology has invading all areas of human life

(industry, environment, agriculture, healthcare, supply chain, etc.) (Bouguettaya et al., 2017). It is proving itself as the best way to connect real and virtual worlds (Rani et al., 2021). Thereby, it enables the fusion between information and the physical world. In the IoT system sensors, actuators and other uniquely identifiable objects are interconnected. They interact between them through the Internet. Thus, it allows to these objects to offer their functionalities in the form of services (Kahloul et al., 2019). These IoT services are developed to allow real-time observation and control of particular phenomena in a specific space or object of everyday life (Razzaque et al., 2016). In daily life, the user requirements are complex. In order to ensure them, new services are obtained by composing atomic ones. These services are called Composite

* Corresponding author.

E-mail address: boucetti.rabah30@gmail.com (R. Boucetti).

Peer review under responsibility of King Saud University.



Production and hosting by Elsevier

Services. These latter provide new functionalities where an atomic services cannot provide individually (Lemos et al., 2015). In fact, the IoT composite services enables the aggregation of intelligent object services to meet complex needs (Chen et al., 2016).

The vast proliferation of smart objects generates IoT services with the same functionalities but are distinguished by their Qualities of Service (QoS) properties (Botta et al., 2016). This makes the satisfaction of the user requirements often complex. Indeed, respecting the QoS constraints (user preferences in terms of QoS) is a challenge due to the high number of candidate services for the composition (Jatoth et al., 2018). This challenge consists of selecting the most appropriate services so that the composite service must meet both the functional and non-functional (QoS) requirements of the user (Boucetti et al., 2020; da Silva et al., 2020).

Generally, the QoS-aware IoT services composition process is achieved in four steps (cf. Fig. 1): (1) development of the composition plan, (2) services discovery, (3) services QoS-based selection, and (4) execution and control of the composition. In this process, we can distinguish between two types of services: concrete and abstract services. A Concrete Service (CS) is an invocable one. However, an Abstract Service (AS) denotes a class of concrete services having the same functionality but differ in their QoS attributes (Chai et al., 2021). The IoT concrete services are characterized by their Quality of Service (QoS) metrics. These latter allow to the user to be able to understand and express its requirements using these metrics.

In the services composition process, there are two categories of approaches that take into account the QoS (Alrifai and Risse, 2009): (1) Approaches based on *local selection*. They allow to determine, individually, the optimal concrete service with respect to QoS for each abstract service of the composition plan. Although, they have a reasonable time complexity; these approaches cannot guarantee that a composite service meets user-dictated global QoS constraints. (2) Approaches based on *global optimization* in which the QoS constraints are taken into account at the level of the composite service. The most obvious way is to evaluate all the possible compositions, then determine the best one (most optimal) in terms of QoS by respecting the imposed global constraints. This leads us to an exponential complexity problem (NP-hard).

To deal with the above-mentioned challenges, this paper proposes an approach based on Genetic Algorithms and Neural Networks for QoS-aware IoT Services Composition (GNN-QSC). It is based on global QoS optimization for IoT services composition. Since our approach is a global optimization based, to find the optimal composition is an NP-hard problem. So, the idea is to reformu-

late the composition problem into a classification problem in order to reduce the search space as well as the composition time. Indeed, we use the neural network model to classify the concrete services, of each participating abstract service in the composition plan, into different categories according to their QoS level. After that, the genetic algorithm is used to find the near-optimal IoT service composition.

In the literature, some studies have addressed some of these challenges. Among the proposed solutions, we can cite: approaches based on Genetic Algorithms (GA) (Gao et al., 2014; Kashyap et al., 2020a; Li et al., 2014; Yang and Li, 2015), those based on GA and Particle Swarm Optimization (PSO) (Kashyap et al., 2020b; Liu et al., 2013), and Cuckoo-Inspired ones (Kurdi et al., 2018). Other solutions that have attempted to reduce the composition space by removing candidate services that are not promising in terms of QoS. For this, clustering techniques were used to group services according to their relevance in the composition process and take into consideration only the relevant clusters (Khanouche et al., 2019a, 2019b; Tong et al., 2020; Wang et al., 2020).

Compared to the above works, our proposition takes into consideration both the optimization of the global QoS constraints, the search space and time. Therefore, our contribution allows to solve the problem of the IoT services composition while satisfying the global QoS constraints. We can summarize it as follows:

- We cluster Concrete Services (CS), belonging to the same Abstract Service (AS) (services having the same functionalities), into QoS-level clusters. To do this, we propose a neural network model to evaluate IoT services and to group them according to the appropriate QoS level. The number of neurons in the input layer corresponds to the QoS attributes vector dimension. The output layer contains the four classes (Platinum, Gold, Silver or Bronze).
- We decompose each QoS attribute, for each Abstract Service (AS_i), into M levels of QoS from the minimum to maximum values of each QoS attribute. So that, we will obtain M theoretical services (M theoretical QoS vectors) for each abstract service.
- We apply the proposed Genetic Algorithms (GA), on the obtained theoretical abstract services, to obtain the Ideal Theoretical Composition (ITC). This latter satisfies the overall QoS constraints and, has qualities of service almost optimal.
- We evaluate the atomic theoretical services that compose the Ideal Theoretical Composition (ITC) by the neural network model. The latter makes it possible to give as a result, for each abstract service, the category of concrete services to be engaged in the composition. To this end, for each atomic theoretical service in the ITC, we select only the concrete services having the same category of this service in the corresponding abstract service. This allows reducing the workspace.
- We apply the second proposed GA to obtain the quasi-optimal concrete composition. This GA is applied on the initial population whose individuals are random feasible compositions formed from previous selected services.
- To validate the effectiveness of the proposed work, we have performed several simulations. The obtained results show the effectiveness of our dynamic IoT services composition under global QoS constraints in a large-scale environment. In addition, the proposed work improves both the search space and time compared to the other works.

The remainder of this paper is organized as follows: Section 2 provides an overview of related work on the QoS-aware IoT services composition. The service and composite service models, the QoS model and global constraints, the utility functions as well as the formalization of the service composition problem under global QoS constraints are presented in Section 3. Our approach is

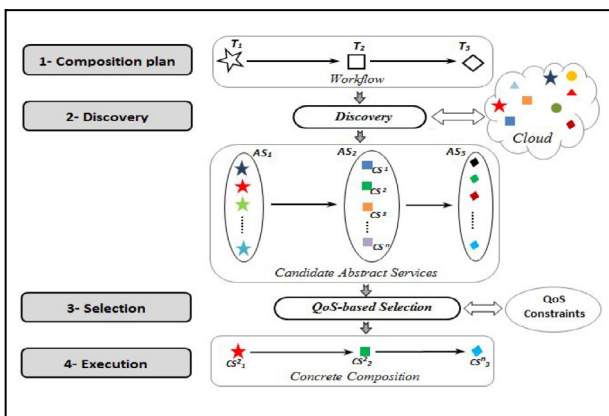


Fig. 1. QoS-aware composition overview.

detailed in Section 4, while the evaluation and discussion are presented in Section 5. Finally, the conclusion and some perspectives are given in Section 6.

2. Related work

In this section, we review the main existing works that tackle the IoT services composition under the QoS constraints.

The resolution of QoS-aware IoT/web services composition problem consists at first to: (1) define a QoS model (QoS vector), (2) define the QoS attributes aggregating rules according to the composition structure, and (3) define the utility function formula allowing the service to be evaluated with respect to its QoS model (da Silva et al., 2020). These points are discussed almost in all works dealing with the QoS-aware services composition, as in (Alrifai and Risse, 2009; Chai et al., 2021; Chen et al., 2017; Jatoth et al., 2018; Kashyap et al., 2020b; Khanouche et al., 2019a, 2019b, 2016; Li et al., 2014; Liu et al., 2016, 2013; Qi et al., 2018; Wang et al., 2020). In this paper, we are inspired from these works to define the QoS model and aggregation rules appropriate to our approach, as well as the function utility based on Simple Additive Weighting (SAW) (Kaliszewski and Podkopaev, 2016). Then, a near-to-optimal composition is chosen from several alternatives based on the QoS-utility function.

To deal with the IoT services composition issue, several solutions have been proposed. Among these solutions, bio-inspired meta-heuristic approaches: approaches based on Genetic Algorithms (GA) (Gao et al., 2014; Kashyap et al., 2020a; Li et al., 2014; Yang and Li, 2015), GA and Particle Swarm Optimization (PSO) based approaches (Kashyap et al., 2020b; Liu et al., 2013), and Cuckoo-Inspired approaches (Kurdi et al., 2018).

To address the user QoS constraints, the authors in (Li et al., 2014) have formulated the problem of the IoT Service Composition oriented QoS (QSC) as a Multi-Criteria Goal Programming (MCGP) model, and developed a Multi-population Genetic Algorithm (MGA) to solve it.

The authors in (Liu et al., 2013) propose the cooperation between the Genetic Algorithm (GA) and the Particle Swarm Optimization (PSO) to solve the problem of IoT service composition based on the QoS. According to the authors, the proposed approach takes into account: (i) Proportional evolutionary optimization to ensure the diversity of the population, (ii) better and improved local strategy for candidate service selection, (iii) the best disruptive global strategy along the best overall particle, and (vi) the self-adaptive learning rate tuning mechanism that compromises the speed of convergence and the optimized result; all this, when the service composition is done at a large scale. On the other hand, Kashyap et al., (2020a) made a comparison between the two algorithms GA and PSO for solving the IoT services composition problem. The authors confirm that the two algorithms solve the problem of service composition in IoT. Experimental results on the real-world dataset demonstrate that the solutions obtained using GA more expressive than those from PSO.

Yang and Li (2015) evaluate each IoT service using a Multi-Attribute Decision-Making method (MAMD) and choose the elites in terms of QoS to be part of the composition. An improved Genetic Algorithm (GA), by adding greedy algorithm, is used as the global optimization method. The obtained result is the service composition with the maximum utility function value.

In (Gao et al., 2014), a QoS aggregation scheme is proposed to calculate the overall QoS vector for the composite service. The optimality of a composition is determined using a QoS utility function taking into account user constraints and a weight vector. Finally, a genetic algorithm is developed to create an optimal service composition.

Kashyap et al. (2020b) propose a multi-objective metaheuristic search algorithm to solve the problem of QoS based IoT services composition. The Non-dominated Sorting Genetic Algorithm (NSGA-II) is used to provide optimal composition.

In (Kurdi et al., 2018), the authors develop an algorithm inspired by the behaviour of cuckoo to find existing composite services which meets efficiently user demand in a multi-cloud environment (MCE). The MultiCuckoo algorithm considers the services composition problem and its peculiarities in the Multi-Cloud Environment (MCE). Its evaluations show that it consumes less time and resources.

Other approaches try to reduce the composition space, by removing candidate services that are not promising in terms of QoS. For that, clustering techniques are used to group services according to their relevance in the composition process and take into consideration only the relevant clusters (Khanouche et al., 2019a, 2019b; Tong et al., 2020; Wang et al., 2020).

In (Khanouche et al., 2019a), a Clustering-based and QoS-aware service Composition Algorithm (CQCA) is proposed in an Ambient Intelligence environment (Aml). To this end, the K-means method is used to cluster the services according to QoS level i.e., each cluster represents a QoS level. Using the characteristics of the result clusters, a new utility function is proposed to eliminate services that are not promising in terms of QoS. This technique allows the reduction of space and time of the composition. Then, the lexicographic method is used to verify the satisfaction of global constraints. Finally, a search tree was built to determine the quasi-optimal composition. On another hand, a fast and reliable service composition approach is proposed in (Wang et al., 2020). The approach integrated the physical, cyberspace and social networks. First, the computation of Skyline components is used to prune redundant ones. Then, using the coefficient of variation, the components with high quality of service (QoS) fluctuation are filtered out. Finally, a Mixed Integer Programming (MIP) is used to select the optimal components by maximizing the fitness function according to the end-to-end QoS requirements of users. In the context of large-scale IoT environment, a two-steps QoS-aware services composition algorithm (TS-QCA) is proposed in (Khanouche et al., 2019b). TS-QCA takes into account the global QoS constraint. It is based on clustering and Shuffled Frog Leaping Algorithm (SFLA) methods to reduce the composition time. Thus, improve the algorithm convergence by focusing only on promising candidate services in the composition process.

Another challenge in IoT is optimal energy management as part of IoT services composition (Alsaryrah et al., 2018; Baker et al., 2017; Chai et al., 2021; Khanouche et al., 2016; Tong et al., 2020).

Chai et al. (2021) propose fast energy-centered and QoS-aware service composition approach (FSCA-EQ) for IoT service composition. They use the hierarchical optimization mechanism. To pre-select the candidate service with higher QoS, the Ratio of Compromise (CRM) method is employed. In addition, to select the optimal service as the final composite service, the concept of relative dominance is applied depending on its energy profile, QoS attributes and user preferences. FSCA-EQ strikes a good balance between QoS level and power consumption for composite service in a large-scale IoT environment. Khanouche et al. (2016) have proposed Energy-Centric and QoS-aware IoT services Selection Algorithm (EQSA) for IoT services composition. The services whose qualities of service satisfy the user are pre-selected, by using a lexicographic optimization strategy and a QoS constraint relaxation technique. The problem is seen as a problem of multi-objective optimizations. Minimize energy consumption in order to ensure high availability of composite services while ensuring QoS requirements. The authors in (Tong et al., 2020) combine both clustering techniques and energy management. The traditional QoS model is extended by introducing energy with the proposal of a method

to calculate it. Considering the new QoS model, the global constraints will be decomposed into local QoS constraints for atomic services. The K-means method is used to obtain clusters of atomic services. Finally, the optimal service is selected locally in each cluster. In (Alsaryrah et al., 2018), the authors reduce the problem of finding a balance between the QoS level and the energy consumed, of IoT service composition, to an optimization problem of the shortest path with two objectives. They use the exact algorithm named Pulse to solve the problem. In (Baker et al., 2017), a multi-cloud IoT service composition algorithm called (E2C2) is developed. E2C2 aims to seek and interact as few IoT services as possible in the energy-sensitive composition plane. A formal translation of user needs, transformation modelling and analysis are adopted for the proposed algorithm. A minimum of IoT services satisfying user demand are researched and integrated in an energy efficient manner.

Other approaches use various visions to solve the IoT services composition problem. In (White et al., 2018) an approach for self-management of reliable systems is proposed. It uses collaborative filtering and objective-oriented IoT service composition. Thus, Matrix Factorization (MF) based collaborative filtering is used in a goal-driven and self-managed service model for the execution of tasks in the IoT. The objective-oriented model adapts dynamically based on the prediction of QoS, thus it enables disaster recovery and reducing failures and it lessens re-execution. The authors in (Gharineiat et al., 2021) propose a framework of crowdsourced IoT services to select and compose mobile crowdsourced IoT services. It is based on spatio-temporal factors. To select and compose mobile services an algorithm based on deep reinforcement learning is developed. This latter takes into account QoS parameters

without using an index. To discover services in motion, Spatio-temporal MapReduce based on flock patterns also is developed by using Apache Spark. The proposed approach is scalable and high accurate in comparison with the ground-truth. Zhou and Ma (2013) decompose the complex model of the QoS calculation into four basic models, and attribute to each QoS calculation model an adapted method according to the IoT characteristics. Then, a heuristic backtracking (BT) algorithm is used to meet the QoS requirement. Cuong et al. (2016) present a Service Monitoring and Composition System (SMACS) in environmental sensor networks. Two parameters (reliability and reactivity) are estimated by SMACS using a naive Bayesian classifier. These parameters are used to obtain the optimal services composition. They allow the improvement of the overall QoS of the entire sensors network by excluding faulty nodes.

The QoS-aware IoT service composition approaches presented in this section are summarized in Table 1. They are compared in terms of: (1) resolution principle that represents the used solving method, (2) scalability, (3) composition optimality i.e. compare the obtained composition with respect to the optimal one, (4) energy awareness, and (5) the complexity analysis.

After this literature review, we highlight the contributions and originality of our approach (GNN-QSC) compared to the QoS-aware IoT service composition approaches presented above.

- Several approaches have been proposed to reduce the search space of IoT services composition problem. These approaches use clustering techniques to reduce search space and consequently composition time. They eliminate candidate services or compositions with little promise in terms of QoS

Table 1
Approaches summarize and comparison.

Approach	Comparison criterion	Scalability	Optimality	Energy	Complexity
	Methods used				
(Chai et al., 2021)	CRM + Relative dominance	Yes	Optimal	Yes	Not evaluated
(Khanouche et al., 2016)	Lexicographic optimization + relative dominance	Yes	Close-to-optimal	Yes	$O(n*m) + O(m*p \log p)$
(Khanouche et al., 2019a)	k-means + Utility function + Lexicographic optimization methods	Yes	Near-to-optimal	No	$O(m*n*k*p*t) + O(m*n) + O(m*r)$
(Wang et al., 2020)	Skyline component + QoS fluctuation + MIP algorithm	Not evaluated	Not evaluated	No	Not evaluated
(Khanouche et al., 2019b)	Clustering + SFLA	Yes	Near-to-optimal	No	Not evaluated
(Li et al., 2014)	MCGP model + MGA	Yes	Optimal	No	Not evaluated
(Liu et al., 2013)	GA + PSO	Not evaluated	Not evaluated	No	Not evaluated
(Kashyap et al., 2020a)	(Cooperative evolution algorithm) GA + PSO Algorithms	Not evaluated	Not evaluated	No	Not evaluated
(Yang and Li, 2015)	GA + Greedy algorithm	Not evaluated	Not evaluated	No	Not evaluated
(Gao et al., 2014)	QoS aggregation schema and Utility Function + GA	Yes	Optimal	No	Not evaluated
(Kashyap et al., 2020b)	Non-dominated Sorting GA NSGA-II	Not evaluated	Not evaluated	No	Not evaluated
(Kurdi et al., 2018)	MultiCuckoo algorithm	Not evaluated	Near-to-optimal	No	Not evaluated
(Tong et al., 2020)	Extended QoS model + K-means	Not evaluated	Optimal	Yes	$O(n*m*d)$
(Alsaryrah et al., 2018)	Pulse Algorithm	Yes	Optimal	Yes	Not evaluated
(Baker et al., 2017)	E2C2 Algorithm	Not evaluated	Optimal	Yes	Not evaluated
(White et al., 2018)	Matrix Factorization	Not evaluated	Not evaluated	No	Not evaluated
(Gharineiat et al., 2021)	Deep Reinforcement Learning + Spatio-temporal MapReduce	Yes	Optimal	Yes	Not evaluated
(Zhou and Ma, 2013)	BackTracking Algorithm	Yes	Sub-optimal	Yes	Not evaluated
(Cuong et al., 2016)	Naive Bayesian Classifier	Not evaluated	Not evaluated	No	Not evaluated
The proposed approach GNN-QSC	Neural Networks + Genetic Algorithm	Yes	Near-to-optimal	No	$O(P_s) + O(n)$

(Khanouche et al., 2019a, 2019b; Tong et al., 2020; Wang et al., 2020). Unlike to techniques adopted in these approaches, mainly the k-means method, the proposed neural network model can be used to determine the category of an IoT service without retraining it. Thus, it allows us to improve the execution time.

- The use of the genetic algorithm is more benefit to compose IoT services than the other methods. Indeed, Kashyap et al., (2020a) made a comparison between the two algorithms GA and PSO used to solve the IoT services composition problem. The authors demonstrate that the solutions obtained using GA is more expressive than those obtained from PSO.
- During the composition process, bio-inspired meta-heuristic based approaches may engage inappropriate services (Gao et al., 2014; Kashyap et al., 2020b, 2020a; Kurdi et al., 2018; Li et al., 2014; Liu et al., 2013; Yang and Li, 2015). To deal with these limitations, we propose, in this paper, the QoS-intervals decomposition. This latter consists to decompose the QoS-intervals into theoretical levels that generate theoretical services in each abstract service. Moreover, from these theoretical services we search the Ideal Theoretical Composite service (ITC) with a high level of QoS. Thus, we can guarantee that for each abstract service, we retain only the adequate services in the concrete composition. For that, we compare them with the categories of the theoretical services that constitute the ITC.
- The majority of the aforementioned approaches specify the QoS attributes to be studied (such as execution time, cost, availability, etc.), which makes them restricted approaches. However, our approach is general and flexible. Therefore, it can be applied to any QoS model.

3. Background, definitions and models

In this section, we will present the context as well as the necessary definitions in order to facilitate and simplify the comprehension of the proposed approach.

3.1. Dynamic IoT services composition model

The dynamic model of IoT services composition is illustrated through Fig. 2. The same shape indicates same services functionalities and the different colors indicate the differences in terms of

QoS. Thus, the user queries a directory containing N service classes (the tasks T_i) to select m tasks ($m \leq N$). These tasks are structured in a given order (composition structure), in order to satisfy a business process needs. Each task T_i ($i \leq m$) is represented by a class of services, called Abstract Service (AS). It contains n identical IoT services, in term of their functional properties, but different in term of their non-functional properties (QoS). The dynamic IoT services composition consists to select the optimal service for each task (Abstract Service AS) at execution time. Such, the composite service result satisfies the global QoS constraints and provides the optimal overall QoS.

3.2. Definitions

To understand the proposed approach, we will give some useful definitions:

- **Concrete Service (CS)** is an instance of an abstract service. It is represented by a tuple $CS = [F, I, O, QoS]$, where: I/O is respectively the vectors of input and output parameters of a CS. F is a function that will transform the inputs I into outputs O . It represents the functional properties of the service CS. On another side, QoS is the quality of service attributes vector (the concrete service non-functional properties).
- **Abstract Service (AS)** is a set of n similar concrete services (CS) in term of their functionality, $AS_i = \{CS_i^1, CS_i^2, CS_i^3, \dots, CS_i^n\}$. Thus, the $CS_i^{k \in [1..n]}$ have the same functionality (F), the same inputs and outputs (the vectors I, O), but their QoS attributes (vector QoS) are different.
- **Abstract Composite Services (ACS)** is a set of m abstract services, $ACS = \langle AS_1, AS_2, \dots, AS_m \rangle$. They are connected, to each other, by one of the service composition structures: sequential, parallel, conditional, loop or a combination of these structures. The ACS is formulated according to the user functional requirements by using a set of connected abstract services.
- **Concrete Composite Service (CCS)** is the instantiation of the Abstract Composite Service. The instantiation is done by the invocation for each abstract service AS_i the appropriate concrete service CS_i^k ($i \in [1..m], k \in [1..n]$), $CCS = \langle CS_1^k, \dots, CS_m^k \rangle$.
- **QoS Attributes Vector of a CS** noted $CS[QoS]$. It is defined by the QoS attributes vector. Formally, $CS_i^k[QoS] = [qos_{i1}^k, \dots, qos_{ip}^k, \dots, qos_{iq}^k]$, where $i \in [1..N], k \in [1..n], p \in [1..q]$, N is the abstract services number in the directory, n is the concrete services number in the abstract service AS_i and q is the QoS attributes number. The qos_{ip}^k is the value of the p^{th} QoS attribute of the k^{th} concrete service that belongs to the i^{th} abstract service. We distinguish between two types of QoS attributes: (1) *negative attributes* (cost, response time, etc.) which have a negative effect on the level of the QoS, i.e., more their values are higher more the QoS is lower. (2) *Positive attributes*, such as availability, reliability, etc., which have a positive effect on the level of the QoS, i.e., more their values are higher more the QoS is better. For this, we need to assign to each concrete service a utility function fu .
- **QoS Attributes Vector of a CCS** is defined as follows: $CCS.QoS = [Q_1, \dots, Q_q]$, where q is the QoS attributes number and Q_p is the value of the p^{th} attribute after aggregation. The aggregation of attributes depends on the used function (Summation, Product, Maximum or Minimum) and on the composition structure (Sequential, Parallel, Conditional and Loop). Table.2 summarizes the different aggregation functions of m services. P_p is the probability to select a concrete service in the conditional composition structure. And, Nbr is the number of iterations in the loop structure (Khanouche et al., 2019a). However, parallel conditional and loop structures can be transformed into a sequential model as presented in (Zeng et al., 2004). In the same way, a utility function FU is associated with each Concrete Composite Service calculated from its aggregated QoS values (Khanouche et al., 2019a).

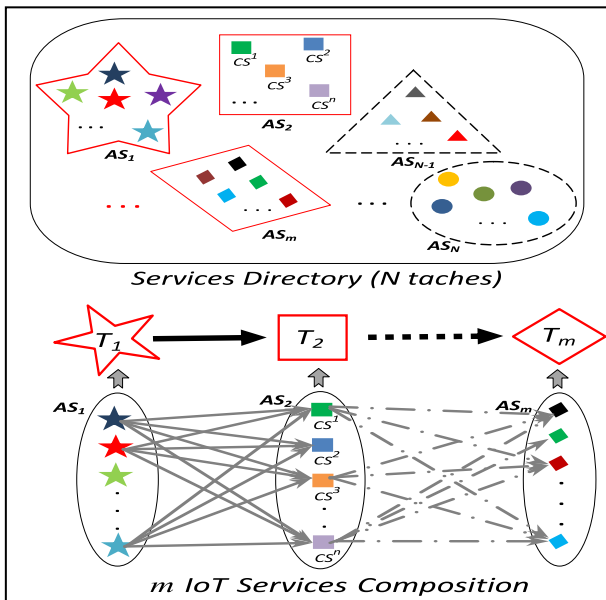


Fig. 2. Dynamic IoT services composition process.

Table 2

QoS-Aggregation functions (Khanouche et al., 2019a).

Function	Composition structure			
	Sequential	Parallel	Conditional	Loop
Summation	$\sum_{p=1}^m qos_{ip}^k$	$\sum_{p=1}^m qos_{ip}^k$	$\sum_{p=1}^m P_p * qos_{ip}^k$	$Nbr * qos_{ip}^k$
Product	$\prod_{p=1}^m qos_{ip}^k$	$\prod_{p=1}^m qos_{ip}^k$	$\sum_{p=1}^m P_p * qos_{ip}^k$	$(qos_{ip}^k)^{Nbr}$
Maximum	$\sum_{p=1}^m qos_{ip}^k$	$\max_{1 \leq p \leq m} qos_{ip}^k$	$\sum_{p=1}^m P_p * qos_{ip}^k$	$Nbr * qos_{ip}^k$
Minimum	$\min_{1 \leq p \leq m} qos_{ip}^k$	$\min_{1 \leq p \leq m} qos_{ip}^k$	$\min_{1 \leq p \leq m} qos_{ip}^k$	qos_{ip}^k

- **Utility Function of a CS**, in order to evaluate a concrete service CS_i^k by using its QoS vector, a utility function fu is associated to it. The role of this function is to transform all the values of its QoS vector into a single value. This latter allows the comparison between services that belong to the same Abstract Service (AS) in term of QoS. In this paper, we use the *Simple Additive Weighting technique* (SAW). It is widely adopted, in the literature, in the field of service composition (da Silva et al., 2020). The utility calculation involves:

- The normalization of the QoS ($Nqos$) attributes values in values between 0 and 1, by comparing each value of QoS with the maximum and minimum values of this attribute. This allows making them independent from the units and ranges of measurements. However, as mentioned above, positive attributes should be maximized, while negative attributes should be minimized. Thus, the normalization will be done in two different ways, as defined in (1):

$$Nqos_{ip}^k = \begin{cases} 1, \text{If } (qos_p^{\max} - qos_p^{\min}) = 0 \\ \text{Else} \\ \begin{cases} \frac{qos_p^{\max} - qos_{ip}^k}{qos_p^{\max} - qos_p^{\min}} \text{ if } \text{is a negative attribute} \\ \frac{qos_{ip}^k - qos_p^{\min}}{qos_p^{\max} - qos_p^{\min}} \text{ if } \text{is a positive attribute} \end{cases} \end{cases} \quad (1)$$

Where qos_{ip}^k represents the current value of the p^{th} QoS attribute of the k^{th} concrete service CS_i^k in the abstract service AS_i . Moreover, $qos_p^{\max}$ and $qos_p^{\min}$ represent, respectively, the maximum and minimum values of the p^{th} QoS attribute.

- The weighting to represent priorities and preferences of the user. w_p , (such as: $\sum w_p = 1, p = 1..q$) are weights assigned to each QoS attribute. The differences in weights represent the user preferences.
- Finally, the utility of the concrete service is obtained by the weighted sum of the normalized QoS values, as given in formula (2):

$$fu(CS_i^k) = \sum_{p=1}^q w_p * Nqos_{ip}^k \quad (2)$$

- **Utility Function of a CCS**, the utility of a given Concrete Composite Service $CCS = \langle CS_1^k, \dots, CS_i^k, \dots, CS_m^k \rangle$ is calculated, as in the utility function of a concrete service, by mapping the values of the QoS vector aggregated $[Q_1, \dots, Q_p, \dots, Q_q]$ into a single value by using the same technique SAW. Sun and Zhao (2012) have proved that the global utility of a composite service is the sum of the local utilities of the concrete services forming the composition. It is given by formula (3).

$$FU(CCS) = \sum_{i=1}^m fu(CS_i^k) \quad (3)$$

- **Global QoS Constraints**. The user's QoS preferences are formulated as global constraints. These latter are imposed on the values of the QoS attributes of the composition (after aggregation). So, the Global QoS constraints is: $CG(QoS) = [Cg_1, \dots, Cg_q]$; where q is the attributes number and Cg_p is the global constraint imposed by the user on the p^{th} attribute of the composite service Q_p . It can be a lower bound ($Q_p \geq Cg_p$), if the attribute is a positive attribute, or an upper bound ($Q_p \leq Cg_p$) in the case where the attribute is negative.
- **Feasible, Near-Optimal CCS**. We consider that a concrete composition (CCS) is feasible if: for each abstract service ($AS_i, i = 1..m$), in the composition, a single concrete service (CS_i^k) is chosen. In addition, the aggregated QoS values of the composition (Q_p) satisfy the overall QoS constraints (Cg_p) imposed by the user (i.e. $Q_p \geq Cg_p$ if the attribute is positive, $Q_p \leq Cg_p$ otherwise) (Khanouche et al., 2019a).
- **Concrete Composite Service (CCS)** is called a near-optimal one if it is feasible and has best overall utility ($\max(FU)$) (Khanouche et al., 2019a).

4. Proposed approach

One of the main challenges in the IoT environment is IoT services composition. Smart things, sensors and actuators are aggregated together to achieve a suitable composited service for user's requests (Asghari et al., 2018). Several domains are concerned by the IoT service composition. The review made by Asghari et al. (2019), indicates that the smart city is the most dominant field in the use of the IoT services composition, followed by health-care, environmental and industrial applications. The same review confirms that the QoS-aware compositions are the most widely used approaches in these application areas. Therefore, our proposed work may be applied to the above-cited domains.

This section is composed of two parts. The first one is devoted to the general description of our approach (GNN-QSC). However, the second one consists of an illustrative scenario to show the functionality steps of the proposed approach.

4.1. General description

Depending on the creation time of the composite service, we can distinguish two kinds of service composition, *static* composition and *dynamic* composition. In a static composition, the aggregation of services is done in the design phase. This composition mode is suitable for cases where a fixed and unchanged number of features to be composed can satisfy the user's request. The static composition is not adequate in the event that changes are expected at runtime, such as replacing an unavailable service or where there is a better alternative service. On another side, a dynamic composition allows the selection and aggregation of services during execution process. This is done automatically by exploiting semantic web techniques and artificial intelligence. Dynamic composition is the ideal mode for composing IoT services; this is due to the natural dynamicity of IoT environment. However, the dynamic composition of IoT services presents several challenges such as establishing the ideal composition that satisfies the user's QoS preferences. In order to address this challenge, we propose our approach based on Genetic algorithms and Neural Networks for QoS-aware IoT Services Composition (GNN-QSC). It leads to an almost optimal composition of IoT services while respecting the QoS constraints imposed through the user's request.

In this paper, the IoT services composition process consists of choosing from each abstract service AS_i ($i = 1..m$), belonging to the composition plan (Abstract Composite Service ACS), the appropriate concrete service CS_i^k in order to maximize the overall QoS gain of the concrete composite service CCS. This is realized while respecting the QoS preferences (global constraints) imposed by the user ($Q_p \leq Cg_p$ if the qos_p attribute is a negative attribute that one wishes to minimize, $Q_p \geq Cg_p$ if the qos_p attribute is a positive attribute to be maximized).

First, we develop a neural network model that allows to evaluate each IoT service through its QoS attributes vector; then to assign it to a category of quality of service level. The IoT services QoS-attributes are similar to those of web services. However, they are different in terms of QoS attributes related to smart objects such as Power Consumption and Location (Singh and Baranwal, 2018). Therefore, the QWS datasets¹ can be used (Al-Masri and Mahmoud, 2007). The QWS Dataset 1.0 is used for the supervised training of the Neural Network model; while, QWS Dataset 2.0 will be used to evaluate the effectiveness of our approach. For the neural network training phase, we use the QWS Dataset 1.0 that contains services decreed by the QoS attribute vectors (input vector) with its evaluation (labels having as values: bronze, silver, platinum or gold).

Second, we decompose for each attribute the interval $[qos_p^{min}, qos_p^{max}]$ into M QoS-levels (where qos_p^{min} , qos_p^{max} are, respectively, the minimum and the maximum value of the p^{th} attribute, $p = 1..q$ and q is the attributes number).

Third, we use the M theoretical services, generated in the previous step (the new QoS vectors after decomposition into M levels), to make the Ideal Theoretical Composition (ITC) by applying the genetic algorithm. The theoretical services that compose the ITC will be evaluated with the neural network model. This is in order to deduce the categories of the real concrete services for each abstract service in the composition.

Finally, the genetic algorithm is used, once again, to get the needed optimal composition. Fig. 3 illustrates an overview of the proposed approach.

Clustering techniques allow to group concrete services (CS), which belongs to an abstract service (AS) (services having the same functionalities), according to their QoS parameters into a set of clusters called QoS levels. Each level groups services that have similar QoS attributes values. In this paper, we propose a neural network model to evaluate IoT services and to group them according to the appropriate QoS level. We have chosen the neural network as a classifier because it gives correct predictions for new situations based on the knowledge acquired during the learning process (Al-Masri and Mahmoud, 2009, 2006). In addition, solution parameters are generated iteratively.

Our neural model contains only one hidden layer of neurons. These latter use the sigmoid activation function. The number of neurons in the input layer corresponds to the QoS attributes vector dimension (q neurons). The output layer contains the number of classes we want to have. It is appropriate to the classes' number in the dataset used in the supervised learning process of our neural network. As mentioned above, we will use QWS Dataset 1.0 that represents the measurements of nine QoS attributes for 365 real services. They are classified into four different types: Platinum, Gold, Silver or Bronze. Our model should be able to classify a given service into one of these classes. The Platinum class contains services having a higher level of quality (less response time, less price, high availability, etc.). However, the Bronze one will contain services having lower quality (high response time, high price, low

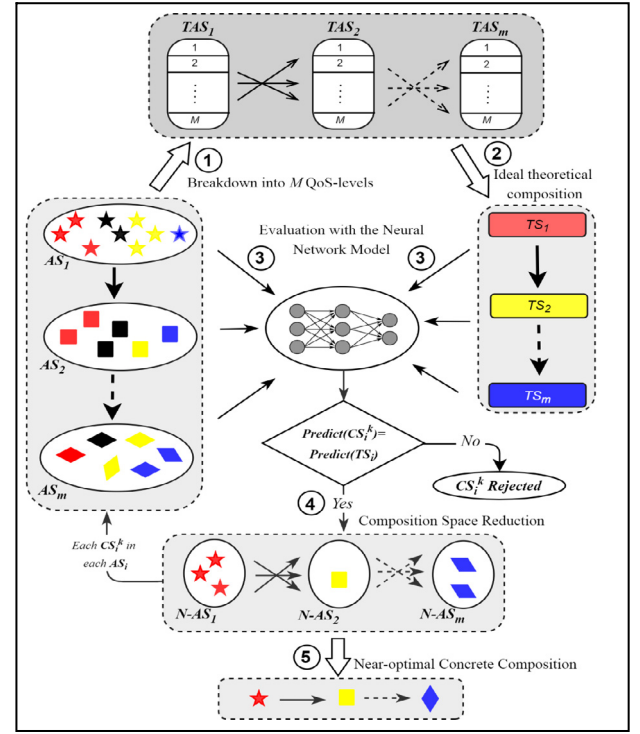


Fig. 3. GNN-QSC overview.

availability ...). Fig. 4 illustrates an overview of our neural network.

4.1.1. QoS-Intervals breakdown into M levels

In this step, we decompose for each abstract service (AS_i), and for each QoS attribute (qos_{ip} , $p = 1..q$) the interval $[qos_{ip}^{min}, qos_{ip}^{max}]$ into M QoS levels using formula (4). Thus, we can form $M-1$ equal sub-intervals of QoS for each attribute; qos_{ip}^{max} and qos_{ip}^{min} are, respectively, the greater and the smallest values of the p^{th} attribute in the QoS vectors of all the concrete services belonging to the abstract service AS_i . M is a constant number that is fixed a priori. In order to ensure a balanced distribution of real values of qos_{ip} in the formed sub-intervals, we choose M according to the distribution degree of qos_{ip} values in the interval $[qos_{ip}^{min}, qos_{ip}^{max}]$.

$$qosth_{ip}^l = qos_{ip}^{min} + \frac{l-1}{M-1} * (qos_{ip}^{max} - qos_{ip}^{min}) \quad (4)$$

$$1 \leq l \leq M, 1 \leq p \leq q, 1 \leq i \leq m$$

We do the same operation for all the attributes (q attributes) and for each abstract service in the composition plane (m abstract services). We will obtain M new vectors of the theoretical QoS values ($qosth_{ip}^l$). These latter are included between the values of qos_{ip}^{min} and qos_{ip}^{max} of each abstract service, which we consider as the M Theoretical Services TS_i^l ($l = 1..M$) of the Theoretical Abstract Service TAS_i .

4.1.2. Find the ideal theoretical composition

At the end of the previous step, we obtain a m theoretical abstract services (TAS_i , $i = 1..m$), containing theoretical services (TS_i^k , $k = 1..M$). These latter are obtained from concrete services belonging to abstract services (AS_i , $i = 1..m$) of the composition plane. We apply a heuristic method based on Genetic Algorithms (GA) to obtain the ideal theoretical composite service with the best theoretical QoS performance.

In GA, solutions must be represented as individuals, and the fitness function evaluates the relevance of each individual. After the

¹ The QWS Dataset, <https://qwsdata.github.io/>, Last update: 01 November 2019.

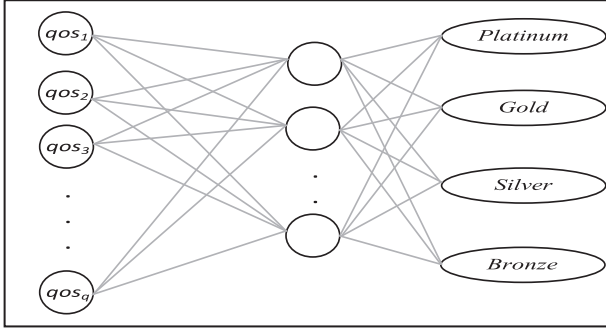


Fig. 4. Neural network model.

creation of the initial population, we apply the genetic operators (selection, crossover and mutation), that are personalized to our objectives, to develop compositions and find the best one.

• Individuals coding

An individual (a chromosome) represents a feasible composition, i.e. a composition that satisfies the overall constraints. An individual is formed by concatenating the theoretical QoS vectors (*qosth*) of the Theoretical Services (TS) of each Theoretical Abstract Service (TAS), according to the abstract composition plan, as shown in Fig. 5.

In our approach, each individual is composed of $m \cdot q$ genes (m is the number of tasks to be composed and q is the number of QoS attributes). Each gene contains a theoretical QoS value resulting from the step of decomposing into M QoS-levels.

• Initial Population

The initial population is formed by a set of valid compositions (individuals) chosen in a random manner; i.e. it selects from each theoretical abstract service (TAS) a candidate theoretical service (TS) for the composition. Then, by using the aggregation formulas shown in Table.2, we calculate the aggregate QoS values of this individual and check if they meet the overall QoS constraints. Otherwise, that individual will be eliminated and another one will be built. We repeat this process until we obtain the desired number of valid individuals that forms the initial population.

• Fitness Function

As an individual represents a valid service composition, the evaluation function must, at first, verify the satisfaction of the global QoS constraints after calculating the aggregated QoS values. Then, it calculates the fitness of the chromosome with the utility function FU assigned to the composite service in formula (3).

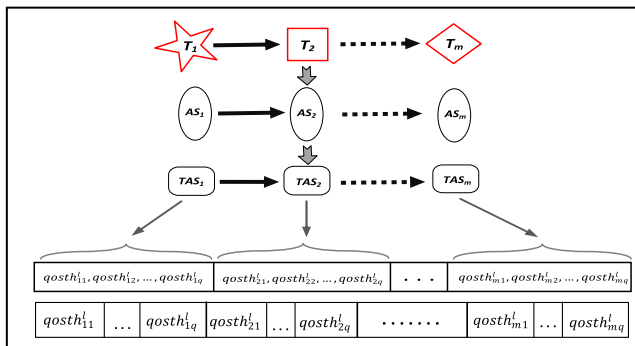


Fig. 5. Individuals Coding.

• Genetic Operators

In our approach, we select the two individuals that have the highest values of the fitness function (elitist method) in order to select two parents to produce the descendants. This will be done in each generation. We adopt the multipoint crossover method to exchange information between the two parents that produce two descendants. The crossing points are chosen, randomly, for each theoretical service of the composition. In addition, the descendants are generated by exchanging the genes of the segments between these points two by two.

For the mutation operator, we try to generate better individuals in each generation by maximizing their fitness functions; this can be obtained by optimizing both the positive and negative attributes. For this, we choose, randomly, for each service of the composition three genes (QoS attributes). If the chosen gene is a positive attribute then the mutation will be done with a randomly generated value. This latter will be between the maximum and the current value of this attribute, in the abstract service, that belongs to the related service. Otherwise, i.e. in the case where it is a negative attribute, the mutation will be done with a randomly generated value between the minimum and the current value of this attribute.

4.1.3. Evaluation with the neural model

After convergence of the genetic algorithm, we obtain a composition of m theoretical services satisfying the overall QoS constraints and, having qualities of service almost optimal. Assuming $ITC = \langle TS_1, TS_2, \dots, TS_m \rangle$ is the ideal theoretical composition. Our approach evaluates each theoretical service TS_i ($i = 1..m$) by the neural network model ($Predict(TS_i)$). It allows to know, as results, their categories (Platinum, Gold, Silver or Bronze). By the same way, we can find the categories of concrete services in each abstract service using the same neural network.

4.1.4. Composition space reduction

The QoS vectors forming ITC are theoretical (*qosth*). In reality, they are used to infer the category of concrete services in each abstract service to be engaged in the optimal concrete composition. Thus, we eliminate the inappropriate services that reduce the workspace. We will retain only the concrete services that have the same category of the corresponding theoretical service in each abstract service. This allows to reduce the search space to three quarters in each abstract service. (cf. algorithm.1).

Inputs: $ACS = \langle AS_1, AS_2, \dots, AS_m \rangle$
 // Abstract Composite Service (m Tasks)
 $AS_i = \{CS_i^1, CS_i^2, \dots, CS_i^{n_i}\}$
 // Abstract Service i (contains n_i concrete services)
 $ThC_{ideal} = \langle TS_1, TS_2, \dots, TS_m \rangle$
 // Ideal theoretical composition

```

1: for  $i = 1$  to  $m$ 
2:    $N-AS_i \leftarrow \emptyset$ 
3:   for  $k = 1$  to  $n_i$ 
4:     if  $Predict(CS_i^k) = Predict(TS_i)$  then
5:        $N-AS_i \leftarrow N-AS_i \cup CS_i^k$ 
6:     end if
7:   end for
8: end for

```

Outputs: $N-ACS = \langle N-AS_1, N-AS_2, \dots, N-AS_m \rangle$

4.1.5. Search the Near-Optimal concrete composition

To obtain the near-optimal composition, the genetic algorithm is used for the second time. It allows reducing the population (obtained result at the previous steps). The individuals of the initial population are, randomly, formed by the concatenation of concrete services of the new abstract services ($N-AS_i, i = 1..m$). The individuals are feasible composite services. We maintain the same genetic algorithm policy previously applied with regard to the application of genetic operators. Once the genetic algorithm is converged, the result is a composition of the candidate concrete services that satisfy the overall QoS constraints with near-optimal utility.

The complexity of the GNN-QSC approach depends on: m, n, q and M where: m is the number of abstract services which form an abstract composite service; n is the number of concrete services per abstract service; q is the number of QoS attributes; and M is the QoS-levels number. To find the overall approach complexity, we must calculate that of each its phases. The QoS-intervals breakdown complexity is of $O(M)$. The complexity of the first GA to find the Ideal Theoretical Composition is of $O(P_s * q * m)$, where P_s is the population size and $q * m$ is the chromosome size (genes number). The evaluation with the neural model has a time complexity of $O(m)$. As for the composition space reduction phase, its complexity is of $O(m * n)$. Finally, the complexity of the second GA is $O(P_s * m)$. Therefore, the overall complexity of the proposed approach is $O(M) + O(P_s * q * m) + O(m) + O(m * n) + O(P_s * m)$. As $M \ll n, m \ll n$ and $q \leq 9$, this complexity becomes $O(P_s) + O(n)$.

4.2. Illustrative scenario

In order to illustrate the steps that must be followed according to our approach (GNN-QSC), we propose the following scenario: we assume that the user's composite request (ACS) contains two tasks (AS), $ACS = \langle T_1, T_2 \rangle$. For each task, there are nine concrete IoT services with different QoS vectors. These vectors are chosen randomly from the QWS Dataset 2.0 (Al-Masri and Mahmoud, 2007). The QWS Dataset 2.0 includes a set of 2507 services; each row, in this dataset, represents a real service with its corresponding nine QoS measurements. The assumed services are described in Table 3.

The global QoS constraints are supposedly the following: CG (QoS) = [Response Time ≤ 9790.17 ms, Availability $\geq 2\%$, Throughput ≥ 5.72 invokes/second, Successability $\geq 4.42\%$, Reliability $\leq 44.32\%$, Compliance $\geq 30.47\%$, Best Practices $\geq 26\%$, Latency ≤ 6233.67 ms, Documentation $\geq 0.04\%$]. Response Time,

Reliability and Latency are negative attributes; the six others are positive ones. For the sake of simplicity, we also consider that all nine QoS attributes have the same weight for the user ($w_p = 1/9, p = 1.0.9$).

The problem to be solved is how to choose in each abstract service, the appropriate concrete service. Thus, we can obtain a composite service with a maximum of QoS-utility and which respects the imposed global QoS-constraints.

In order to achieve our goals, the proposed approach (GNN-QSC) consists, as a first, to build and train the neural model using TensorFlow 2.3.0 library. The neural model allows evaluating each IoT service through its QoS attributes vector; then to assign it to a category of QoS-level. Our neural model is composed of a single hidden layer containing 32 neurons. These neurons use the sigmoid activation function. The first layer contains 9 neurons corresponding to the nine QoS attributes studied in the training dataset QWS 1.0. The output layer contains four neurons that correspond to the services classes (Platinum, Gold, Silver and Bronze) of the same dataset. The softmax activation function is used for the output layer neurons. The supervised training is done on 80% of the QWS Dataset 1.0, while the remaining 20% are used for the test. The model has given an accuracy of 91.33%. We recall that the trained phase is done only once.

The second phase begins once the first phase is achieved. It is composed of five steps that are as follows:

4.2.1. QoS-Intervals breakdown into m QoS-Levels

This step allows us to reduce the search space, especially in a large-scale environment. We consider M as a constant number which is fixed a priori such as $M \ll n$. In our scenario $n = 9$, we choose $M = 4$. Afterwards, by using formula (4) we will obtain four theoretical services in each abstract service, as shown in Table 4.

4.2.2. Find the ideal theoretical composition

By using the Genetic Algorithm, we look for the ideal theoretical composition from the theoretical services generated in the previous phase (Table 4). The individuals of the initial population are formed by concatenating randomly the theoretical QoS vectors (qosth) of the theoretical services (TS), of each theoretical abstract service (T), two by two ($\langle TS_{12}, TS_{24} \rangle, \langle TS_{13}, TS_{21} \rangle \dots$). At this step, we check at each composition the satisfaction of the global QoS-constraints. We use formulas of Table 2 to make the appropriate QoS attributes aggregations (see Section 3.2) for the composite service. The evaluation of individuals (theoretical compositions) will

Table 3
IoT Services descriptions.

AS	CS	QoS Attributes								
		RespTime	Availability	Throughput	Successability	Reliability	Compliance	BestPractices	Latency	Documentation
T_1	T_{11}	107.57	80	1.7	81	67	78	82	18.21	61
	T_{12}	269.83	85	4.5	86	53	89	66	74.96	6
	T_{13}	42.5	72	13.5	72	73	78	84	2.5	11
	T_{14}	106.75	90	16.2	96	73	78	80	24.13	93
	T_{15}	408.21	56	5.0	58	73	78	80	121.46	89
	T_{16}	103.25	93	20.0	98	73	78	84	1.0	86
	T_{17}	114.33	85	16.2	95	73	100	84	13.0	39
	T_{18}	40.0	72	19.8	72	73	78	84	3.5	5
	T_{19}	672.2	91	7.9	97	73	100	84	9.2	36
T_2	T_{21}	408.0	83	15.2	84	83	89	91	3.0	6
	T_{22}	115.0	83	22.3	84	83	89	91	4.0	5
	T_{23}	46.05	89	20.7	96	67	100	77	4.48	8
	T_{24}	499.0	83	19.7	84	83	89	91	31.0	5
	T_{25}	3343.0	83	5.5	84	83	89	91	2.0	3
	T_{26}	109.6	99	19.3	100	73	78	62	1.0	94
	T_{27}	42.15	56	10.4	56	60	89	69	1.32	37
	T_{28}	102.0	90	18.6	97	73	78	80	1.0	92
	T_{29}	38.0	71	21.1	72	73	78	84	2.0	2

be done by the composite service utility function (FU) by using formula (3) via formulas (1) and (2).

After the convergence of the GA, we will obtain a composite theoretical service satisfying the global QoS-constraints with a better utility. It is the Ideal Theoretical Composition (ITC). In our scenario, the obtained ITC is shown in Table 5.

4.2.3. Evaluation with the neural model

In this step, we evaluate the services that compose the ideal theatrical composition, as well as concrete services by using the neural network. The result is shown in Table 6.

4.2.4. Composition space reduction

In this step, we reduce the concert composition space by retaining, only, the concrete services of the *Gold* category from the first Abstract Service (T_{11} , T_{14} , T_{16} , T_{17}), and the *Platinum* category services from the second Abstract Service (T_{23} , T_{27} , T_{29}) according to ITC atomic services categories.

4.2.5. Find Near-optimal concrete composition

After having selected the concrete candidate services in each abstract service, we look for the almost optimal composition by using the genetic algorithm again. The individuals (compositions) are built randomly from the previously selected services. The GA convergence gives us the quasi-optimal concrete composite service i.e. the composition that has a better utility in terms of QoS by satisfying the imposed constraints. The result is: $\langle T_{16}, T_{27} \rangle$ with $UF_{\text{utility}} = 1,098$.

5. Experimental results and evaluation

This step of our work consists to verify the effectiveness of our dynamic IoT services composition under global QoS constraints approach (GNN-QSC) in the large scale environment.

The experimental environment is a PC with the following configurations: CPU: Intel (R) Core (TM) i9 8th Gen, 2.90 GHz; GPU: NVIDIA Quadro P4200; Memory: 16 GB; and OS: Windows 10 64-bits. The approach is carried out with the Python 3.7.9 programming language under the Spyder IDE environment.

The experimental context is designed as follows: we suppose that there is an IoT service composition process. It is composed of m abstract services (tasks). Each abstract service has n concrete

Table 6

Services evaluation with the neural model.

AS	CS Evaluation		ITC Services Evaluation	
T_1	T_{11}	Gold	TS_1	Gold
	T_{12}	Silver		
	T_{13}	Platinum		
	T_{14}	Gold		
	T_{15}	Silver		
	T_{16}	Gold		
	T_{17}	Gold		
	T_{18}	Platinum		
	T_{19}	Silver		
T_2	T_{21}	Silver	TS_2	Platinum
	T_{22}	Gold		
	T_{23}	Platinum		
	T_{24}	Silver		
	T_{25}	Bronze		
	T_{26}	Gold		
	T_{27}	Platinum		
	T_{28}	Gold		
	T_{29}	Platinum		

candidate services. To build these abstracts services, we randomly choose, at each time, n services from the QWS dataset 2.0. To give credibility to our assessment, we have retained the nine global QoS constraints used in the illustrative scenario (Section 4.2).

As indicated previously, the Genetic Algorithm (GA) is used twice. The goal of the first is to obtain the ideal theoretical composition and, to deduce there from the categories of concrete services in each abstract service. The second GA is performed on the concrete services of the retained categories to obtain the near-optimal concrete composition. The GA configuration is as follows:

- GA Parameters setting:

Generally, the key parameters of GA should be determined by experiments. The result of the existing research proves that the best value for population size is between 20 and 160. The best crossover probability value is between 0.25 and 1.0, and the mutation probability value is between 0.005 and 0.1 (Liu et al., 2016). In this paper, we set the size of the population to 20, the crossover probability to 1.0, and the parameter of mutation to 0.05. For the stop criterion, we chose to stop the algorithm when the population

Table 4
QoS-Intervals breakdown.

TAS	TS	qaosth values								
		ReTimeTh	AvailTh	ThroughTh	SuccessabTh	ReliabTh	ComplTh	BestPractTh	LatencyTh	DocumentTh
T_1	TS_{11}	40	56	1,7	58	53	78	66	1	5
	TS_{12}	250,73	68,33	7,8	71,33	59,66	85,33	72	41,15	34,33
	TS_{13}	461,46	80,66	13,9	84,66	66,33	92,66	78	81,30	63,66
	TS_{14}	672,2	93	20	98	73	100	84	121,46	93
T_2	TS_{21}	38	56	5,5	56	60	78	62	1	2
	TS_{22}	1139,66	70,33	11,1	70,66	67,66	85,33	71,66	11	32,66
	TS_{23}	2241,33	84,66	16,7	85,33	75,33	92,66	81,33	21	63,33
	TS_{24}	3343	99	22,3	100	83	100	91	31	94

Table 5
Ideal Theoretical Composition (ITC).

TS	qosth values								
	ReTimeTh	AvailTh	ThroughTh	SuccessabTh	ReliabTh	ComplTh	BestPractTh	LatencyTh	DocumentTh
TS_1	109.23	90.67	15.33	73.77	67.12	75.98	82.45	0.73	91.03
TS_2	50.59	60.76	9.34	71.33	64.29	85.12	69.59	2.78	39.54

cannot evolve any more, i.e. when the difference between the utility of the solution in the generation g and $g + 1$ become very small (in our case, $FU(g) - FU(g + 1) = 0.0001$).

- Generating initial population:

For the first GA, the initial population is defined by the decomposition of each abstract service into M QoS-levels by applying formula (4) (section 4.1.1). After obtaining the theoretical abstract services, we can produce the chromosomes (valid service compositions). We use a two-dimensional array to store a chromosome. In the first row, we store the valid theoretical composition. This latter is randomly generated from the theoretical services in each theoretical abstract service, according to the composition plan. The generated composition satisfies the overall QoS constraints. In the second row, we store the value of its Utility Function that is calculated through formula (3). Thereby, we repeat the chromosome generation method until we obtain 20 individuals.

Before we use the second GA, we must, at first, classify all the concrete services of each abstract service by using the proposed neural model. After that, for each abstract service we retain the services having the same categories of the atomic theoretical services that compose the ITC. This latter is found in the previous step. The retained services are used to build individuals of the initial population for the second GA. Each chromosome is a composition of the previously retained services. These latter are generated randomly so that the composition meets the overall QoS constraints. The process is repeated until we reach 20 individuals.

- Selection Operation:

We adopt the elitist selection as a selection operation in both GAs. This selection technique gives, always, individuals with best scores (utility). First, we calculate the Function Utility (FU) of each chromosome (composition) by using the utility function defined in equation (3). Then, the chromosomes are classified according to their utilities. The two best parents are chosen, at each time, for the children reproduction.

- Crossover Operation:

Crossover is an important operator in the generation of new individuals. It interchanges parts of both parents with some probability. In this paper, we adopt the gene segment crossing method. For the first GA, in each service constituting the two parents, we choose, randomly, a crossing point (an integer between 1 and 9). Then services segments are exchanged two by two between the parents according to the crossing point. However, in the second GA, we will consider a service as a gene, and the crossing point will be chosen randomly between one and m (m is the number of services to be composed). Then, the two segments of both parents are swapped.

- Mutation Operation:

The mutation is also a method that allows to generate new individuals by modifying one or more genes on a chromosome. For the first GA, the mutation is applied on three genes (three attributes) chosen randomly in each service of the theoretical composition. Since the objective is to maximize the utility function, we behave as follow: if the chosen attribute (gene), for the mutation, is a positive attribute then we replace its value with another one, which is generated randomly between the current value and the maximum of this attribute in the abstract service. On the other hand, if the chosen attribute is a negative one, we replace its value with a value

generated randomly between the minimum of this attribute in the abstract service and its current value.

In the second GA, each gene is a concrete service with its utility. To make the mutation, we also choose randomly a service and replace it with another among the concrete services belonging to the same abstract service.

To verify the feasibility in a large scale environment and the effectiveness of the proposed method, we carry out five simulation experiments as indicated in the following sub-sections.

5.1. Experiment 1

In this experiment, we will evaluate the approach (GNN-QSC) according to the number of QoS-levels. For this, we assume that there are three abstract services in the composition plan ($m = 3$). Each abstract service contains thirty ($n = 30$) concrete services. The values of the algorithm execution time are obtained according to the different values of M $M \in \{5, 10, 15, 20, 25, 30\}$, where $M \leq n$. The experimental results are shown in Fig. 6. The vertical axis represents the execution time of the algorithm, and the horizontal one indicates the M values.

As shown in Fig. 6, when the number of QoS-level M has increased, the execution time of the service composition has not, significantly, increased (it is almost identical). So, the two great values 2.493 s and 2.498 s are obtained when M is equal to 5 and 30 respectively, whereas the minimum value 1.805 s is obtained when $M = 10$, which represents the best execution time. Therefore, we will set M to 10 in the next experiments.

5.2. Experiment 2

The aim of this experiment is to verify the influence of the number of tasks, to be composed, on the composition method performance. To this end, we set M to 10 (QoS-levels), n to 40 (number of concrete services in each AS), and we vary the number of abstract services (m) between 3 and 7. We compare the obtained results (execution times) of our approach with those of the optimal compositions. The experimental results are illustrated in Fig. 7.

As shown in Fig. 7, the execution time of the GNN-QSC increases with the increase of tasks number to be composed. It increases from 2.215 s to 8.325 s. This increase is very low but acceptable compared to that of the optimal composition approach which was exponential. Fig. 7 shows clearly the efficiency of GNN-QSC, which reduces, considerably, the services composition time.

5.3. Experiment 3

In order to assess the performance and efficiency of GNN-QSC approach, simulation scenarios are carried out to compare it with

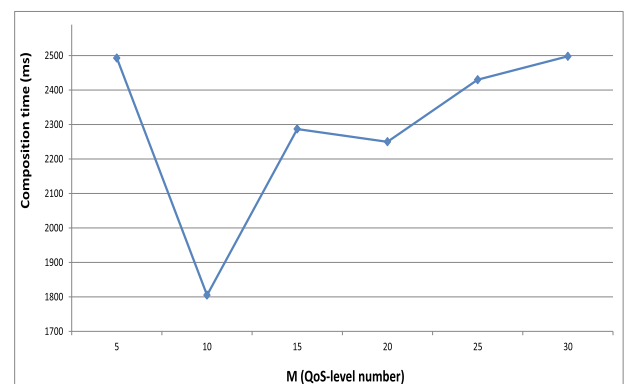


Fig. 6. GNN-QSC execution time with different QoS-levels.

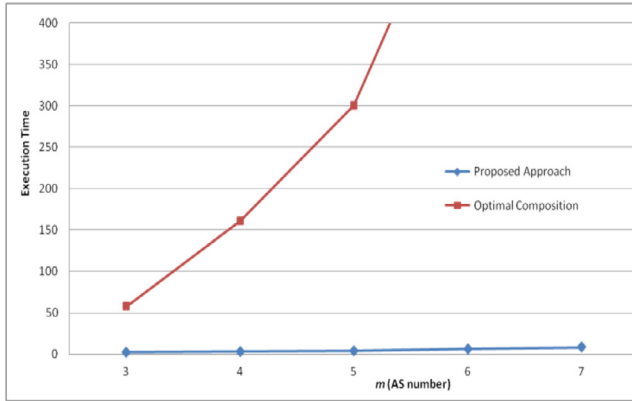


Fig. 7. GNN-QSC execution time with different number of AS.

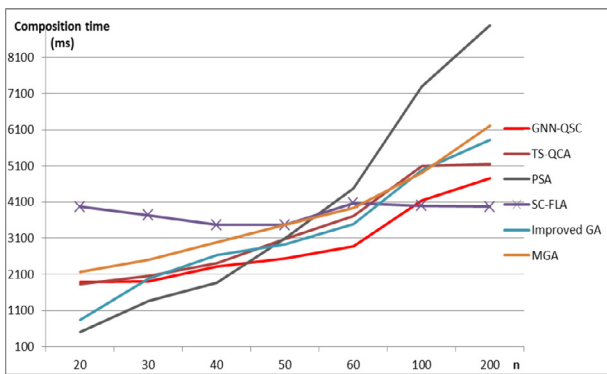


Fig. 8. Impact of the CS's number on the execution time.

five approaches. These latter are: the Two-Step QoS service Composition Algorithm (TS-QCA), approach using shuffled Frog Leaping Algorithm (SC-FLA), the Pareto-based partial Services selection Approach (PSA) (Khanouche et al., 2019b), the Improved binary coded Genetic Algorithm (Improved GA) (Yang and Li, 2015) and Multi-population Genetic Algorithm (MGA) (Li et al., 2014). The comparison indicator is the composition time, i.e. the necessary time to find the near-to-optimal composition. For this, we assume that there are three abstract services in the compositional plane ($m = 3$), and $n \in \{20, 30, 40, 50, 60, 100, 200\}$. Then, we record the execution times average of fifty simulations for each approach. The results are shown in Fig. 8.

Fig. 8 shows that with the increase in the number of concrete services in each abstract service, the composition time does not increase sharply for both algorithms GNN-QSC and TS-QCA. Generally, SC-FLA has a stable composition time, in the large-scale environment. It has the lowest runtime followed by GNN-QSC. However, the other approaches (PSA, MGA and Improved GA) have a composition time which increases considerably with the increase of the CS's number. Overall, the GNN-QSC approach has the best composition time compared to that obtained with the other approaches. This is due to the efficiency of the neural network model employed as a non-promising services eliminating method. It reduces the composition space and consequently the composition time.

5.4. Experiment 4

In this experiment, the GNN-QSC approach will be compared with the TS-QCA, SC-FLA, Improved GA and MGA approaches. The comparison metric is the hypervolume (HV) indicator. The HV indi-

cator is the most widely used metric among several to compare the performance of evolutionary multi-objective algorithms (Guerreiro et al., 2020, Li and Yao, 2019). HV is a unary metric that calculates the volume of the area enclosed by the solution set and a reference point, where a higher value indicates a better result. The authors in (Riquelme et al., 2015) claim that HV covers accuracy, diversity and cardinality, being the only unary metric with this capability. The comparison results are illustrated in Fig. 9.

From Fig. 9, we can see that GNN-QSC and TS-QCA have the best and almost the same HV value with slight superiority of TS-QCA. This means that GNN-QSC and TS-QCA have almost the same performing solution sets. However, the HV values confirm that GNN-QSC has much better performing solution set compared to SC-FLA, Improved GA and MGA approaches.

5.5. Experiment 5

In this last experiment, we will compare the composition optimality of the GNN-QSC, TS-QCA, SC-FLA, Improved GA, MGA and PSA algorithms. The composition optimality is the ratio between the value of the utility function of the near-optimal composition and its optimal value. We assume that there are three abstract services in the compositional plane ($m = 3$), and $n \in \{20, 30, 40, 50, 60, 100, 200\}$. Then, we calculate, for each approach, the optimality average for about fifty simulations. The results are shown in Fig. 10.

As we can see in Fig. 10, the GNN-QSC algorithm is better than the SC-FLA, Improved GA and MGA algorithms in terms of compositional optimality. This is justified by the use of the neural model that allows us to focus only on the adequate services in terms of QoS. However, the optimality of TS-QCA is slightly superior to that of GNN-QSC. The optimality of the PSA algorithm is the ideal one,

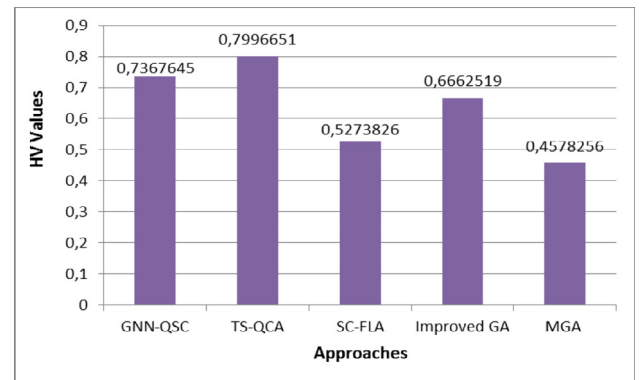


Fig. 9. Hypervolume indicator comparison.

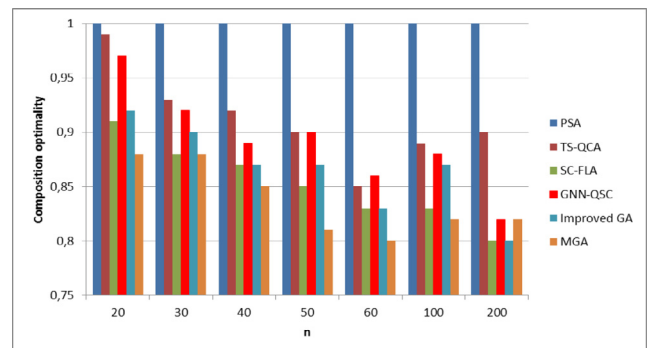


Fig. 10. Optimality against the concrete services number.

since it uses the concept of Pareto dominance to find the optimal composition. However, GNN-QSC is a meta-heuristic method, based on genetic algorithms, to find the near-to-optimal composition in a very short time.

6. Conclusion

Recently, with the considerable advances in the Internet of Things and cloud IoT, the composition of IoT services is an attractive research topic, in particular the QoS-aware composition. Many researchers have addressed this problem. However, a reliable dynamic services composition that can satisfy the global QoS constraints remains an open issue. This paper presents a new approach to the reliable dynamic IoT services composition under global QoS constraints. It is based on evolutionary algorithms and neural networks (GNN-QSC). According to our approach, in order to reduce the number of candidate services, we have proposed an Ideal Theoretical Composition that meets the global QoS-constraints. This optimal theatrical composition is obtained by decomposing the QoS-intervals in each Abstract Service and, by using an improved Genetic Algorithm. For each abstract service, we have performed the proposed clustering method based on a neural network, to predict the category of the concrete services to be retained for the concrete composition. This allows to extract the better concrete services to be engaged in the final composition according to the optimal theoretical composition found previously. Thus, it enables the reduction of the search space and execution time. Then, the second adaptive GA is invoked on the retained concrete services. This is in order to find the near-optimal concrete service composition. The experimental results show the effectiveness of our approach. However, by focusing on the IoT composite service QoS-optimality, some limitations of this work can be noted. For example, some features are not covered in this work but are essential in IoT environments. We can cite managing trust, protecting user privacy preferences and mobility of smart devices with location properties.

In the future, we will study the performance of Artificial Intelligence methods. We will try to propose a new model based on deep learning in order to predict, in a more precise way, the categories of concrete services to be composed. Moreover, we will study how to breakdown the QoS-intervals in order to make our approach more efficient. Actually, we are studying the feasibility to assess the proposed QoS-aware IoT service composition method in a real IoT cloud platform. This is by using QoS-vectors of real IoT services taking into account attributes that are specific to IoT domain, such as energy consumption and location.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

- Al-Masri, E., Mahmoud, Q.H., 2009. Discovering the best web service: A neural network-based solution. *Conf. Proc. - IEEE Int. Conf. Syst. Man Cybern.*, 4250–4255 <https://doi.org/10.1109/ICSMC.2009.5346817>.
- Al-Masri, E., Mahmoud, Q.H., 2007. QoS-based discovery and ranking of Web services. *Proc. - Int. Conf. Comput. Commun. Networks, ICCCN*, 529–534. <https://doi.org/10.1109/ICCCN.2007.4317873>.
- Al-Masri, E., Mahmoud, Q.H., 2006. A context-aware mobile service discovery and selection mechanism using artificial neural networks. *ACM Int. Conf. Proc. Ser.*, 594–598 <https://doi.org/10.1145/1151454.1151467>.
- Alrifai, M., Risse, T., 2009. Combining global optimization with local selection for efficient QoS-aware service composition. *WWW'09 - Proc. 18th Int. World Wide Web Conf.*, 881–890 <https://doi.org/10.1145/1526709.1526828>.
- Alsaryrah, O., Mashal, I., Chung, T.Y., 2018. Bi-objective optimization for energy aware internet of things service composition. *IEEE Access* 6, 26809–26819. <https://doi.org/10.1109/ACCESS.2018.2836334>.
- Asghari, P., Rahmani, A.M., Javadi, H.H.S., 2019. Internet of things applications: A systematic review. *Comput. Networks* 148, 241–261. <https://doi.org/10.1016/j.comnet.2018.12.008>.
- Asghari, P., Rahmani, A.M., Javadi, H.H.S., 2018. Service composition approaches in IoT: A systematic review. *J. Netw. Comput. Appl.* 120, 61–77. <https://doi.org/10.1016/j.jnca.2018.07.013>.
- Baker, T., Asim, M., Tawfik, H., Aldawsari, B., Buyya, R., 2017. An energy-aware service composition algorithm for multiple cloud-based IoT applications. *J. Netw. Comput. Appl.* 89, 96–108. <https://doi.org/10.1016/j.jnca.2017.03.008>.
- Botta, A., De Donato, W., Persico, V., Pescapé, A., 2016. Integration of Cloud computing and Internet of Things: A survey. *Futur. Gener. Comput. Syst.* 56, 684–700. <https://doi.org/10.1016/j.future.2015.09.021>.
- Boucetti, R., Hemam, S.M., HIOUAL, O., 2020. QoS-aware IoT services composition: a survey, in: *To Appear in Distributed Sensing and Intelligent Systems Proceedings of ICDSIS 2020 Conference*. Agadir, Morocco.
- Bouguettaya, A., Singh, M., Huhns, M., Sheng, Q.Z., Dong, H., Yu, Q.i., Neiat, A.G., Mistry, S., Benatallah, B., Medjahed, B., Ouzzani, M., Casati, F., Liu, X., Wang, H., Georgakopoulos, D., Chen, L., Nepal, S., Malik, Z., Erradi, A., Wang, Y., Blake, B., Dustdar, S., Leymann, F., Papazoglou, M., 2017. A service computing manifesto: The next 10 years. *Commun. ACM* 60 (4), 64–72. <https://doi.org/10.1145/2983528>.
- Chai, Z.-y., Du, M.-M., Song, G.-Z., 2021. A fast energy-centered and QoS-aware service composition approach for Internet of Things. *Appl. Soft Comput.* 100, 106914. <https://doi.org/10.1016/j.asoc.2020.106914>.
- Chen, F., Li, M., Wu, H., 2017. GACRM: A dynamic multi-Attribute decision making approach to large-Scale Web service composition. *Appl. Soft Comput.* 61, 947–958. <https://doi.org/10.1016/j.asoc.2017.09.022>.
- Chen, I.-R., Guo, J., Bao, F., 2016. Trust management for SOA-based IoT and its application to service composition. *IEEE Trans. Serv. Comput.* 9 (3), 482–495. <https://doi.org/10.1109/TSC.2014.2365797>.
- Cuong, N.T., Dominguez, F., Long, N.T., Touhafi, A., Steenhaut, K., 2016. Service composition with quality of service management in environmental sensor networks. *Int. J. Ad Hoc Ubiquitous Comput.* 23, 216–229. <https://doi.org/10.1504/IJAHUC.2016.10000318>.
- da Silva, A.S., Ma, H., Mei, Y.i., Zhang, M., 2020. A survey of evolutionary computation for web service composition: A technical perspective. *IEEE Trans. Emerg. Top. Comput. Intell.* 4 (4), 538–554. <https://doi.org/10.1109/TETCI.2020.2969213>.
- Gao, F., Curry, E., Ali, M.I., Bhiri, S., Mileo, A., 2014. QoS-aware complex event service composition and optimization using genetic algorithms. *Lect. Notes Comput. Sci. (including Subser. Lect. Notes Artif. Intell. Lect. Notes Bioinformatics)* 8831, 386–393. https://doi.org/10.1007/978-3-662-45391-9_28.
- Gharineiat, A., Bouguettaya, A., Ba-hutair, M.N., 2021. A deep reinforcement learning approach for composing moving IoT services. *IEEE Trans. Serv. Comput.* 1374, 1–14. <https://doi.org/10.1109/TSC.2021.3064329>.
- Guerreiro, A.P., Fonseca, C.M., Paquete, L., 2021. The hypervolume indicator: problems and algorithms. *ACM Comput. Surv.* 54 (6), 1–42.
- Gupta, D., Rani, S., Ahmed, S.H., Verma, S., Ijaz, M.F., Shafi, J., 2021. Edge caching based on collaborative filtering for heterogeneous icn-iot applications. *Sensors* 21, 1–17. <https://doi.org/10.3390/s21165491>.
- Jatoth, C., Gangadharan, G.R., Fiore, U., Buyya, R., 2018. QoS-aware Big service composition using MapReduce based evolutionary algorithm with guided mutation. *Futur. Gener. Comput. Syst.* 86, 1008–1018. <https://doi.org/10.1016/j.future.2017.07.042>.
- Kahloul, L., Benharzallah, S., Aoudia, I., 2019. Service composition approaches for Internet of Things: a review. *Int. J. Commun. Networks Distrib. Syst.* 23 (1), 1.
- Kaliszewski, I., Podkopaev, D., 2016. Simple additive weighting - A metamodel for multiple criteria decision analysis methods. *Expert Syst. Appl.* 54, 155–161. <https://doi.org/10.1016/j.eswa.2016.01.042>.
- Kashyap, N., Kumari, A.C., Chhikara, R., 2020a. Multi-objective Optimization using NSGA II for service composition in IoT. *Procedia Comput. Sci.* 167, 1928–1933. <https://doi.org/10.1016/j.procs.2020.03.214>.
- Kashyap, N., Kumari, A.C., Chhikara, R., 2020b. Service composition in IoT using genetic algorithm and particle swarm optimization. *Open Comput. Sci.* 10, 56–64. <https://doi.org/10.1515/comp-2020-0011>.
- Khanouche, M.E., Amirat, Y., Chibani, A., Kerkar, M., Yachir, A., 2016. Energy-centered and QoS-aware services selection for internet of things. *IEEE Trans. Autom. Sci. Eng.* 13 (3), 1256–1269. <https://doi.org/10.1109/TASE.2016.2539240>.
- Khanouche, M.E., Attal, F., Amirat, Y., Chibani, A., Kerkar, M., 2019a. Clustering-based and QoS-aware services composition algorithm for ambient intelligence. *Inf. Sci. (Nij)* 482, 419–439. <https://doi.org/10.1016/j.ins.2019.01.015>.
- Khanouche, M.E., Mouloudj, S., Hammoum, M., 2019b. Two-steps QoS-aware services composition algorithm for Internet of Things. *ACM Int. Conf. Proceeding Ser.* 10. <https://doi.org/10.1145/3341325.3342017>.
- Kurdi, H., Ezzat, F., Altoaimy, L., Ahmed, S.H., Youcef-Toumi, K., 2018. MultiCuckoo: Multi-cloud service composition using a cuckoo-inspired algorithm for the internet of things applications. *IEEE Access* 6, 56737–56749. <https://doi.org/10.1109/ACCESS.2018.2872744>.
- Lemos, A.L., Daniel, F., Benatallah, B., 2015. Web service composition: A survey of techniques and tools. *ACM Comput. Surv.* 48 (3), 1–41. <https://doi.org/10.1145/2831270>.
- Li, M., Yao, X., 2020. Quality evaluation of solution sets in multiobjective optimisation: A survey. *ACM Comput. Surv.* 52 (2), 1–38. <https://doi.org/10.1145/3300148>.

- Li, Q., Dou, R., Chen, F., Nan, G., 2014. A QoS-oriented Web service composition approach based on multi-population genetic algorithm for Internet of things. *Int. J. Comput. Intell. Syst.* 7, 26–34. <https://doi.org/10.1080/18756891.2014.947090>.
- Liu, J., Chen, Y., Chen, X., Ding, J., Chowdhury, K.R., Hu, Q., Wang, S., 2013. Kooperativna evolucija za kvalitetno pružanje usluga u paradigmi Interneta stvari. *Automatika* 54, 438–447. <https://doi.org/10.7305/automatika.54-4.417>.
- Liu, Z.Z., Chu, D.H., Jia, Z.P., Shen, J.Q., Wang, L., 2016. Two-stage approach for reliable dynamic Web service composition. *Knowl.-Based Syst.* 97, 123–143. <https://doi.org/10.1016/j.knosys.2016.01.010>.
- Qi, L., Yao, W., Chang, J., 2018. A large scale transactional service selection approach based on skyline and ant colony optimization algorithm. *IEEE/IFIP Netw. Oper. Manag. Symp. Cogn. Manag. a Cyber World, NOMS 2018* 1–7. <https://doi.org/10.1109/NOMS.2018.8406250>.
- Rani, S., Koundal, D., Kavita, Ijaz, M.F., Elhoseny, M., Alghamdi, M.I., 2021. An optimized framework for WSN routing in the context of industry 4.0. *Sensors* 21 (19), 6474. <https://doi.org/10.3390/s21196474>.
- Razzaque, M.A., Milojevic-Jevric, M., Palade, A., Clarke, S., 2016. Middleware for internet of things: A survey. *IEEE Internet Things J.* 3 (1), 70–95. <https://doi.org/10.1109/JIOT.2015.2498900>.
- Riquelme, N., Von Lücken, C., Barán, B., 2015. Performance metrics in multi-objective optimization. In: *Proc. - 2015 41st Lat. Am. Comput. Conf. CLEI 2015*. <https://doi.org/10.1109/CLEI.2015.7360024>.
- Singh, M., Baranwal, G., 2018. Quality of Service (QoS) in Internet of Things. In: *Proc. - 2018 3rd Int. Conf. Internet Things Smart Innov. Usages, IoT-SIU 2018* 1–6. <https://doi.org/10.1109/IoT-SIU.2018.8519862>.
- Sun, S.X., Zhao, J., 2012. A decomposition-based approach for service composition with global QoS guarantees. *Inf. Sci. (Ny)* 199, 138–153. <https://doi.org/10.1016/j.ins.2012.02.061>.
- Tong, E., Chen, L., Li, H., 2020. Energy-aware service selection and adaptation in wireless sensor networks with QoS guarantee. *IEEE Trans. Serv. Comput.* 13 (5), 829–842. <https://doi.org/10.1109/TSC.2017.2749227>.
- Wang, S., Zhou, A.o., Yang, M., Sun, L., Hsu, C.-H., Yang, F., 2020. Service composition in cyber-physical-social systems. *IEEE Trans. Emerg. Top. Comput.* 8 (1), 82–91. <https://doi.org/10.1109/TETC.2017.2675479>.
- White, G., Palade, A., Clarke, S., 2018. QoS prediction for reliable service composition in IoT. *Lect. Notes Comput. Sci. (including Subser. Lect. Notes Artif. Intell. Lect. Notes Bioinformatics)* 10797 LNCS, 149–160. https://doi.org/10.1007/978-3-319-91764-1_12.
- Yang, Z., Li, D., 2015. IoT information service composition driven by user requirement. In: *Proc. - 17th IEEE Int. Conf. Comput. Sci. Eng. CSE 2014, Jointly with 13th IEEE Int. Conf. Ubiquitous Comput. Commun. IUCC 2014, 13th Int. Symp. Pervasive Syst.* 1509–1513. <https://doi.org/10.1109/CSE.2014.280>.
- Zeng, L., Benatallah, B., Ngu, A.H.H., Dumas, M., Kalagnanam, J., Chang, H., 2004. QoS-aware middleware for Web services composition. *IEEE Trans. Softw. Eng.* 30, 311–327. <https://doi.org/10.1109/TSE.2004.11>.
- Zhou, M., Ma, Y., 2013. QoS-aware computational method for IoT composite service. *J. China Univ. Posts Telecommun.* 20, 35–39. [https://doi.org/10.1016/S1005-8885\(13\)60252-6](https://doi.org/10.1016/S1005-8885(13)60252-6).